Exercise3_1

HDL Code

```
module exercise3_1(x, y, out1, out2, out3);
        input x, y;
        output out1, out2, out3;

        buf U1(out1, x);
        nand U2(out2, x, y);
        xnor U3(out3, x, y);


endmodule
```

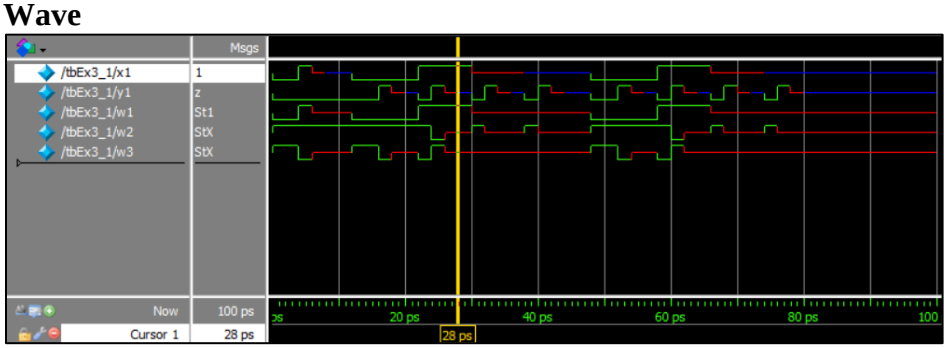
```
module tbEx3_1;
        wire w1, w2, w3;
        reg x1, y1;
        reg monBuff, monNand, monXnor;

        exercise3_1 start3_1(x1, y1, w1, w2, w3);

        initial begin
                monBuff = 1; monNand = 0; monXnor = 0;

                x1=1'b0;y1=1'b0;
                $display("Truth table for buf: ");
                $display("x\tout");

                if(monBuff == 1)
                        $monitor("%b\t%b",x1, w1);

                #2 x1=1'b0;
                #2 x1=1'b1;
                #2 x1=1'bx;
                #2 x1=1'bz;
                #2 $display("\n");
                #2 monBuff = 0; monNand = 1;

                x1=1'b0;y1=1'b0;
                $display("Truth table for nand: ");
                $display("x\ty\tout");

                if(monNand == 1)
                        $monitor("%b\t%b\t%b",x1,y1,w2);
                #2 x1=1'b0; y1=1'b0;
                #2 x1=1'b0; y1=1'b1;
                #2 x1=1'b0; y1=1'bx;
                #2 x1=1'b0; y1=1'bz;

                #2 x1=1'b1; y1=1'b0;
                #2 x1=1'b1; y1=1'b1;
                #2 x1=1'b1; y1=1'bx;
                #2 x1=1'b1; y1=1'bz;

                #2 x1=1'bx; y1=1'b0;
                #2 x1=1'bx; y1=1'b1;
                #2 x1=1'bx; y1=1'bx;
                #2 x1=1'bx; y1=1'bz;

                #2 x1=1'bz; y1=1'b0;
                #2 x1=1'bz; y1=1'b1;
                #2 x1=1'bz; y1=1'bx;
                #2 x1=1'bz; y1=1'bz;
                #2 $display("\n");
                #2 monNand = 0; monXnor = 1;

                x1=1'b0;y1=1'b0;
                $display("Truth table for xnor: ");
                $display("x\ty\tout");

                if(monXnor == 1)
                        $monitor("%b\t%b\t%b",x1,y1,w3);
                #2 x1=1'b0; y1=1'b0;
                #2 x1=1'b0; y1=1'b1;
                #2 x1=1'b0; y1=1'bx;
                #2 x1=1'b0; y1=1'bz;

                #2 x1=1'b1; y1=1'b0;
                #2 x1=1'b1; y1=1'b1;
                #2 x1=1'b1; y1=1'bx;
                #2 x1=1'b1; y1=1'bz;

                #2 x1=1'bx; y1=1'b0;
                #2 x1=1'bx; y1=1'b1;
                #2 x1=1'bx; y1=1'bx;
                #2 x1=1'bx; y1=1'bz;

                #2 x1=1'bz; y1=1'b0;
                #2 x1=1'bz; y1=1'b1;
                #2 x1=1'bz; y1=1'bx;
                #2 x1=1'bz; y1=1'bz;
                #2 $display("\n");

                #18 $finish;
        end
endmodule
```

Transcript

```
VSIM 19> run
# GetModuleFileName: The specified module could not be found.
#
#
# Truth table for buf:
# x     out
# 0     0
# 1     1
# x     x
# z     x
#
#
# Truth table for nand:
# x     y       out
# 0     0       1
# 0     1       1
# 0     x       1
# 0     z       1
# 1     0       1
# 1     1       0
# 1     x       x
# 1     z       x
# x     0       1
# x     1       x
# x     x       x
# x     z       x
# z     0       1
# z     1       x
# z     x       x
# z     z       x
#
```

```
#
# Truth table for xnor:
# x     y       out
# 0     0       1
# 0     1       0
# 0     x       x
# 0     z       x
# 1     0       0
# 1     1       1
# 1     x       x
# 1     z       x
# x     0       x
# x     1       x
# x     x       x
# x     z       x
# z     0       x
# z     1       x
# z     x       x
# z     z       x
#
#
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Gate-Level Modeling/Exercise3_1.v(91)
#    Time: 100 ps  Iteration: 0  Instance: /tbEx3_1
```

Wave



Exercise3_2

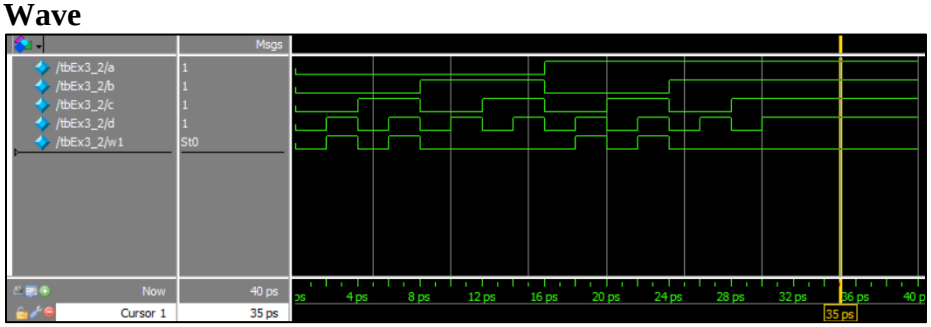
HDL Code

```
module exercise3_2(a, b, c, d, out);
        input a, b, c, d;
        output out;
        wire bNot;

        not U1(bNot, b);
        and U2(out, bNot, d);
endmodule

module tbEx3_2;
        wire w1;
        reg a,b,c,d;

        exercise3_2 start3_2(a,b,c,d,w1);

        initial begin
                $display("Simulating the output of the circuit");
                $display("A\tB\tC\tD\t\tF");
                $monitor("%b\t%b\t%b\t%b\t\t%b",a,b,c,d,w1);

                        a=1'b0; b=1'b0; c=1'b0; d=1'b0;
                #2      a=1'b0; b=1'b0; c=1'b0; d=1'b1;
                #2      a=1'b0; b=1'b0; c=1'b1; d=1'b0;
                #2      a=1'b0; b=1'b0; c=1'b1; d=1'b1;
                #2      a=1'b0; b=1'b1; c=1'b0; d=1'b0;
                #2      a=1'b0; b=1'b1; c=1'b0; d=1'b1;
                #2      a=1'b0; b=1'b1; c=1'b1; d=1'b0;
                #2      a=1'b0; b=1'b1; c=1'b1; d=1'b1;
                #2      a=1'b1; b=1'b0; c=1'b0; d=1'b0;
                #2      a=1'b1; b=1'b0; c=1'b0; d=1'b1;
                #2      a=1'b1; b=1'b0; c=1'b1; d=1'b0;
                #2      a=1'b1; b=1'b0; c=1'b1; d=1'b1;
                #2      a=1'b1; b=1'b1; c=1'b0; d=1'b0;
                #2      a=1'b1; b=1'b1; c=1'b0; d=1'b1;
                #2      a=1'b1; b=1'b1; c=1'b1; d=1'b0;
                #2      a=1'b1; b=1'b1; c=1'b1; d=1'b1;
                #10     $finish;
        end
endmodule
```

Transcript

```
VSIM 6> run
# Simulating the output of the circuit
# A     B       C       D               F
# 0     0       0       0               0
# 0     0       0       1               1
# 0     0       1       0               0
# 0     0       1       1               1
# 0     1       0       0               0
# 0     1       0       1               0
# 0     1       1       0               0
# 0     1       1       1               0
# 1     0       0       0               0
# 1     0       0       1               1
# 1     0       1       0               0
# 1     0       1       1               1
# 1     1       0       0               0
# 1     1       0       1               0
# 1     1       1       0               0
# 1     1       1       1               0
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Gate-Level Modeling/Exercise3_2.v(37)
#    Time: 40 ps  Iteration: 0  Instance: /tbEx3_2
```

Wave



Exercise3_3

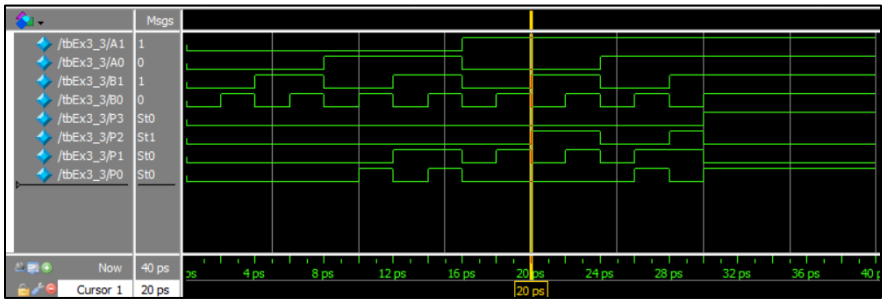
HDL Code

```
module exercise3_3(A1,A0,B1,B0,P3,P2,P1,P0);
        input A1,A0,B1,B0;
        output P3,P2,P1,P0;
        wire W1,W2,W3,W4;

        and U1(P0,A0,B0), U2(W1,A1,B0), U3(W2,A0,B1), U4(W3,A1,B1), U6(W4,W1,W2), U8(P3,W4,W3);
        xor U5(P1,W1,W2), U7(P2,W4,W3);
endmodule

module tbEx3_3;
        reg A1,A0,B1,B0;
        wire P3,P2,P1,P0;

        exercise3_3 multiplier(A1,A0,B1,B0,P3,P2,P1,P0);

        initial begin
                $display("Simulating a 2-bit Multiplier");
                $display("A1\tA0\tB1\tB0\tOutput");
                $monitor("%b\t%b\t%b\t%b\t\%b%b%b%b",A1,A0,B1,B0,P3,P2,P1,P0);

                        A1=1'b0; A0=1'b0; B1=1'b0; B0=1'b0;
                #2      A1=1'b0; A0=1'b0; B1=1'b0; B0=1'b1;
                #2      A1=1'b0; A0=1'b0; B1=1'b1; B0=1'b0;
                #2      A1=1'b0; A0=1'b0; B1=1'b1; B0=1'b1;
                #2      A1=1'b0; A0=1'b1; B1=1'b0; B0=1'b0;
                #2      A1=1'b0; A0=1'b1; B1=1'b0; B0=1'b1;
                #2      A1=1'b0; A0=1'b1; B1=1'b1; B0=1'b0;
                #2      A1=1'b0; A0=1'b1; B1=1'b1; B0=1'b1;
                #2      A1=1'b1; A0=1'b0; B1=1'b0; B0=1'b0;
                #2      A1=1'b1; A0=1'b0; B1=1'b0; B0=1'b1;
                #2      A1=1'b1; A0=1'b0; B1=1'b1; B0=1'b0;
                #2      A1=1'b1; A0=1'b0; B1=1'b1; B0=1'b1;
                #2      A1=1'b1; A0=1'b1; B1=1'b0; B0=1'b0;
                #2      A1=1'b1; A0=1'b1; B1=1'b0; B0=1'b1;
                #2      A1=1'b1; A0=1'b1; B1=1'b1; B0=1'b0;
                #2      A1=1'b1; A0=1'b1; B1=1'b1; B0=1'b1;
                #10     $finish;
        end

endmodule
```

Transcript

```
VSIM 18> run
# GetModuleFileName: The specified module could not be found.
#
#
# Simulating a 2-bit Multiplier
# A1    A0      B1      B0      Output
# 0     0       0       0       0000
# 0     0       0       1       0000
# 0     0       1       0       0000
# 0     0       1       1       0000
# 0     1       0       0       0000
# 0     1       0       1       0001
# 0     1       1       0       0010
# 0     1       1       1       0011
# 1     0       0       0       0000
# 1     0       0       1       0010
# 1     0       1       0       0100
# 1     0       1       1       0110
# 1     1       0       0       0000
# 1     1       0       1       0011
# 1     1       1       0       0110
# 1     1       1       1       1001
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Gate-Level Modeling/Exercise3_3.v(37)
#    Time: 40 ps  Iteration: 0  Instance: /tbEx3_3
```

Wave



Drill3_1

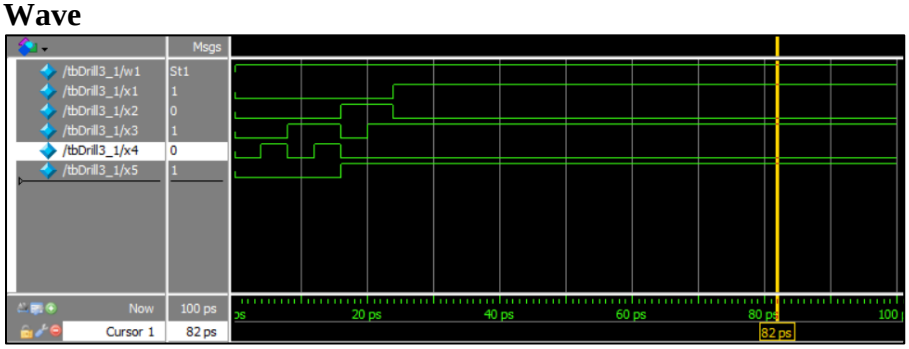
HDL Code

```
module drill3_1(V, W, X, Y, Z, out1);

        input V, W, X, Y, Z;
        output out1;
        wire Vnot, Wnot, Xnot, Ynot, Znot, or1, or2, or3;

        not U1(Vnot, V), U2(Wnot, W), U3(Xnot, X);
        not Ur(Ynot, Y), U5(Znot, Z);
        and U7(or1, Vnot, Wnot, Znot), U8(or2, W, Ynot, Z);
        and U9(or3, V, X, Z);
        or U6(out1, or1, or2, or3);

endmodule

module tbDrill3_1;
        wire w1;
        reg x1, x2, x3, x4, x5;

        drill3_1 test2_1(x1, x2, x3, x4, x5, w1);
        initial begin
                x1=1'b0; x2=1'b0; x3=1'b0; x4=1'b0; x5=1'b0;
                #100 $finish;
        end

        initial begin
                #2 $display(x1,x2,x3,x4,x5," ",w1);
                #2 x1=1'b0; x2=1'b0; x3=1'b0; x4=1'b1; x5=1'b0; //2
                #2 $display(x1,x2,x3,x4,x5," ",w1);
                #2 x1=1'b0; x2=1'b0; x3=1'b1; x4=1'b0; x5=1'b0; //4
                #2 $display(x1,x2,x3,x4,x5," ",w1);
                #2 x1=1'b0; x2=1'b0; x3=1'b1; x4=1'b1; x5=1'b0; //6
                #2 $display(x1,x2,x3,x4,x5," ",w1);
                #2 x1=1'b0; x2=1'b1; x3=1'b0; x4=1'b0; x5=1'b1; //9
                #2 $display(x1,x2,x3,x4,x5," ",w1);
                #2 x1=1'b0; x2=1'b1; x3=1'b1; x4=1'b0; x5=1'b1; //13
                #2 $display(x1,x2,x3,x4,x5," ",w1);
                #2 x1=1'b1; x2=1'b0; x3=1'b1; x4=1'b0; x5=1'b1; //21
                #2 $display(x1,x2,x3,x4,x5," ",w1);
        end
endmodule
```

Transcript

```
VSIM 6> run
# 00000 1
# 00010 1
# 00100 1
# 00110 1
# 01001 1
# 01101 1
# 10101 1
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Gate-Level Modeling/Drill3_1.v(22)
#    Time: 100 ps  Iteration: 0  Instance: /tbDrill3_1
# 1
# Break in Module tbDrill3_1 at C:/intelFPGA_lite/20.1/Gate-Level Modeling/Drill3_1.v line 22
```

Wave



Drill3_2

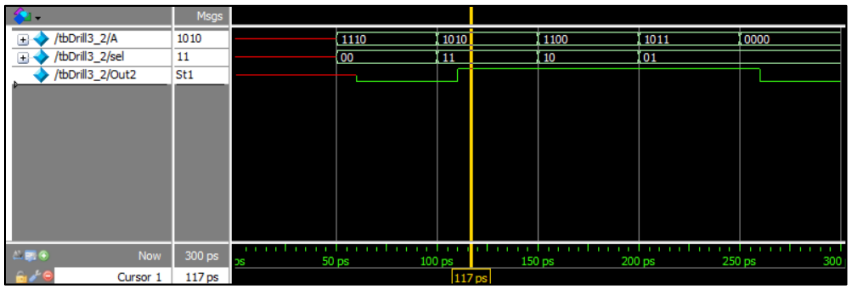
HDL Code

```
module drill3_2(input [3:0]I, input [1:0]S, output Out1);

        wire [3:0]In;
        wire S0not, S1not;
        not #(2) (S0not, S[0]);
        not #(2) (S1not, S[1]);
        and #(5) (In[0], I[0], S0not, S1not);
        and #(5) (In[1], I[1], S[0], S1not);
        and #(5) (In[2], I[2], S0not, S[1]);
        and #(5) (In[3], I[3], S[0], S[1]);
        or #(5) (Out1, In[0], In[1], In[2], In[3]);

endmodule

module tbDrill3_2;

        reg [3:0]A;
        reg [1:0]sel;
        wire Out2;

        drill3_2 t2_2(A, sel, Out2);

        initial begin
                $display(" TIME A select Output");
                $monitor($time,,,,,,,"%h %d %b",A, sel, Out2);
                #50 A=4'hE; sel=2'b00;
                #50 A=4'hA; sel=2'b11;
                #50 A=4'hC; sel=2'b10;
                #50 A=4'hB; sel=2'b01;
                #50 A=4'h0; sel=2'b01;
                #50 $finish;
        end

endmodule
```

Transcript

```
VSIM 3> run
#  TIME A select Output
#                    0      x x x
#                   50      e 0 x
#                   60      e 0 0
#                  100      a 3 0
#                  110      a 3 1
#                  150      c 2 1
#                  200      b 1 1
#                  250      0 1 1
#                  260      0 1 0
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Gate-Level Modeling/Drill3_2.v(31)
#    Time: 300 ps  Iteration: 0  Instance: /tbDrill3_2
```

Wave



Descriptive Explanation

Drill3_1

This code consists of two modules: drill3_1 and tbDrill3_1. The drill3_1 module is a combinational logic circuit that takes five inputs (V, W, X, Y, Z) and produces one output (out1). The module computes the output using a combination of NOT, AND, and OR gates based on the specified logic expressions. The tbDrill3_1 module is a testbench designed to simulate and verify the functionality of the drill3_1 module by applying various input combinations and displaying the results. It initializes the inputs, applies different values in sequence, and prints the values of the inputs and the output to verify the behavior of the circuit.

Drill3_2

This code defines two modules: drill3_2 and tbDrill3_2. The drill3_2 module implements a 4x1 multiplexer using a 4-bit input I and a 2-bit select signal S to produce a single output Out1. The module uses NOT gates to invert the select signals and AND gates to select the appropriate input based on the select signal values, finally using an OR gate to combine the selected inputs into the output. The tbDrill3_2 module is a testbench that simulates the drill3_2 module by applying various combinations of input and select signals, displaying the results with $monitor to observe the output. The testbench runs through several test cases, changing the input and select values every 50-time units, and then finishes the simulation.

Exercise3_1

The provided Verilog code defines a module exercise3_1 that implements three logic gates: a buffer (buf), a NAND gate, and an XNOR gate. The module takes two inputs, x and y, and produces three outputs: out1 from the buffer, out2 from the NAND operation, and out3 from the XNOR operation. The testbench tbEx3_1 simulates the behavior of these gates by iterating through various input combinations, including valid (0, 1) and invalid states (x, z). For each gate, it displays a truth table, using $monitor to print the corresponding outputs while changing the inputs in 2-time unit increments. The simulation aims to comprehensively showcase the output behavior of each gate under different conditions, concluding with a controlled finish after testing all combinations.

Exercise3_2

The provided Verilog code defines a module exercise3_2 that performs a logical operation based on its inputs a, b, c, and d, but only uses b and d to determine the output because of the simplified SOP Logic Expression that equals to (F = B’D). Specifically, it computes the output out as the logical AND of the inverted value of b and the value of d. The testbench module tbEx3_2 initializes a simulation to display the output for all possible combinations of inputs a, b, c, and d. It uses $monitor to continuously print the values of these inputs along with the resulting output during the simulation. The testbench iterates through each input combination in 2-time unit increments, ensuring a comprehensive truth table is displayed for the logical operation defined in exercise3_2.

Exercise3_3

The provided Verilog code defines a 2-bit multiplier module, exercise3_3, which takes two 2-bit binary numbers (A and B) as inputs and produces a 4-bit product (P). The multiplier uses combinational logic with AND and XOR gates to compute the product bits P3, P2, P1, and P0 based on the inputs. The testbench, tbEx3_3, initializes the input signals and sequentially applies all possible combinations of the 2-bit inputs, allowing for a comprehensive simulation of the multiplier's behavior. The $monitor statement displays the current input values along with the corresponding output product in real-time. Finally, the simulation runs for a total of 10-time units, after which it terminates, ensuring all combinations have been tested.